

# A Touchless Gesture-Controlled Media Player Interface

Department of Computer Engineering, Information Technology University (ITU), Lahore, Pakistan

Course: SE305T / MD445T — Human Computer Interaction (Spring 2026)

Rameeza Rahim, Ariba Mumtaz, Saad Ramzan

BSCE22052 BSCE22020 BSCE22013

## Abstract

*Touchscreens and remote controls do not always work well. Sometimes a user's hands are busy, wet, dirty, or just too far from the device. This report describes a touchless media player that uses hand gestures instead of buttons or touch. A normal webcam captures the hand, and Google's MediaPipe library finds 21 key points on it (the fingertips, joints, and wrist). These points are turned into a 63-number feature vector, which is then cleaned up so it does not matter how close the hand is to the camera or where it sits in the frame. We trained three classic machine learning models:  $k$ -Nearest Neighbours ( $k$ -NN), Support Vector Machine (SVM), and Random Forest on a dataset of 5,600 hand-pose samples covering seven gestures, with 800 samples per gesture. The dataset was built programmatically rather than recorded by hand, using pose templates plus random rotation, scaling, and noise to make the samples more realistic. On the test set, SVM gave the best accuracy at 99.54%,  $k$ -NN reached 99.00%, and Random Forest reached 98.86%, though Random Forest was by far the fastest to run (0.02 ms per sample). We also ran a small usability test with five users, who rated the system using the System Usability Scale (SUS). The average score was 82.4 out of 100, which counts as good usability. Overall, the results show that a simple machine learning pipeline, without any deep learning, is enough to build a fast and reliable touchless gesture interface for a media player.*

## I. Introduction

Touchscreens, mice, and remote controls have been the normal way to control media players for years. This works fine when your hands are free. But there are a lot of everyday situations where it does not work so well cooking with messy hands and wanting to pause a video, sitting far from the laptop during a presentation, or simply not wanting to reach for a remote every few minutes. In all these cases, the problem is not the media player itself. The problem is that you have to physically touch something to control it.

Gesture control is one way to fix this. Hands are already a natural way to communicate, and almost every laptop today has a built-in webcam, so no extra hardware is needed. Tools like Google's MediaPipe have also made hand tracking much easier than before it can find 21 points on a hand in real time, even on a normal laptop CPU, without needing a powerful GPU or a special depth camera.

This project looks at whether a simple, classic machine learning approach (not deep learning) can use these MediaPipe hand points to build an accurate and fast gesture-controlled media player. Instead of training a neural network on raw video, which needs a lot of data and computing power, the system here works on a small 63-number feature vector taken from the hand landmarks, and classifies it using standard models from scikit-learn ( $k$ -NN, SVM, and Random Forest). This keeps the system simple, fast to train, and fast enough to run in real time.

The main parts of this report are:

- A complete gesture-to-media-command pipeline covering seven gestures (open palm, fist, thumbs up, thumbs down, point left, point right, and victory sign) mapped to seven commands (play/pause, stop, volume up, volume down, previous track, next track, mute).

- A normalized 63-number feature vector built from MediaPipe's 21 hand landmarks, plus a synthetic dataset of 5,600 samples used to train the models.
- A comparison of three classifiers — k-NN, SVM, and Random Forest — on accuracy, precision, recall, F1-score, ROC-AUC, and speed, checked with 5-fold cross-validation.
- A small usability study with five users, using SUS scores and simple task ratings to check if the system is actually easy to use.

The rest of this report is organised as follows. Section II looks at related work. Section III explains the system design and methodology. Section IV shows the results. Section V covers the usability study. Section VI concludes the report.

## II. Related Work

Gesture-based interaction is not a new idea. Earlier systems used data gloves or basic computer vision to track hands, mainly to avoid the need for physical contact with a device. Interest in touchless interfaces has grown a lot in recent years, especially for public kiosks, shared devices, and medical settings where touching a screen is not always practical or hygienic.

Hand tracking. Older gesture systems often relied on coloured gloves, depth cameras like the Kinect, or custom-trained neural networks running directly on video frames. These needed either special hardware or large labelled datasets and a GPU. MediaPipe changed this by offering a free, pretrained model that can return 21 hand landmarks in real time on a normal laptop. Because MediaPipe already solves the hard part (finding the hand), the remaining classification problem becomes much simpler, which is the approach used in this project.

Classical ML for gesture classification. Other work has shown that once a hand is reduced to landmark points, simple classifiers like k-NN, SVM, and Random Forest can perform very well, without needing deep learning at all. This matches a wider trend in pose-based recognition, where geometric features (joint positions) are usually easier to classify than raw pixels. This project follows the same idea, MediaPipe handles the hand detection, and a classic classifier handles the gesture label.

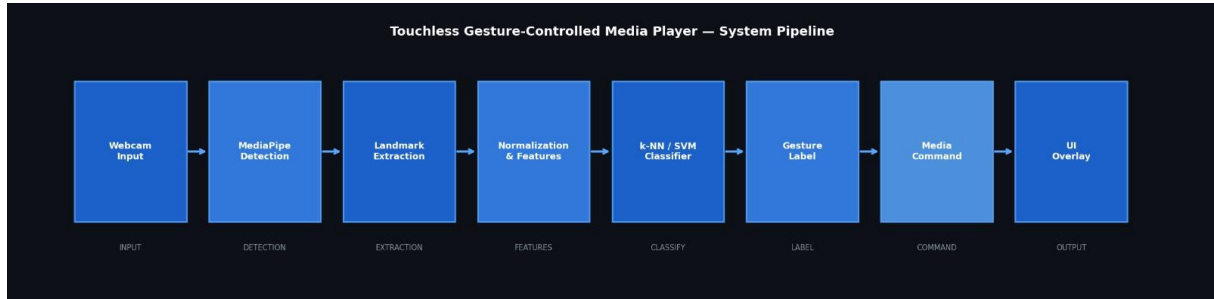
Gesture control for media and devices. Gesture control has already been used for media players, smart TVs, and car infotainment systems, usually with a small set of easy gestures (open hand, fist, pointing, thumbs up/down) mapped to simple commands like play, pause, or volume. Keeping the gesture set small and obvious, rather than large like sign language, makes the system easier to learn and more reliable — which is exactly the goal for a media-control use case.

Usability of gesture interfaces. Past HCI research makes clear that good accuracy alone is not enough — users also need clear, real-time feedback about what the system thinks they are doing, especially in a touchless system with no physical click or vibration to confirm an action. The System Usability Scale (SUS) is a common way to measure this, with scores above about 80 generally seen as good usability. This project follows the same approach: showing the detected gesture, the resulting command, and a confidence score live on screen, and using SUS to evaluate the final interface.

## III. Proposed System and Methodology

### A. System Overview

Fig. 1 shows the overall pipeline. A webcam frame is captured and sent to MediaPipe's Hand Landmarker, which finds a single hand and returns 21 3D landmarks: the wrist, finger joints, and fingertips. These landmarks are turned into a normalized 63-number feature vector, which the trained classifier then turns into one of seven gesture labels. Each label is mapped to a media command (for example, Open Palm means Play/Pause), and the command is sent as a simulated keypress to the system. The detected gesture, the command, and the model's confidence are all shown live on screen, so the user can see what the system is doing.

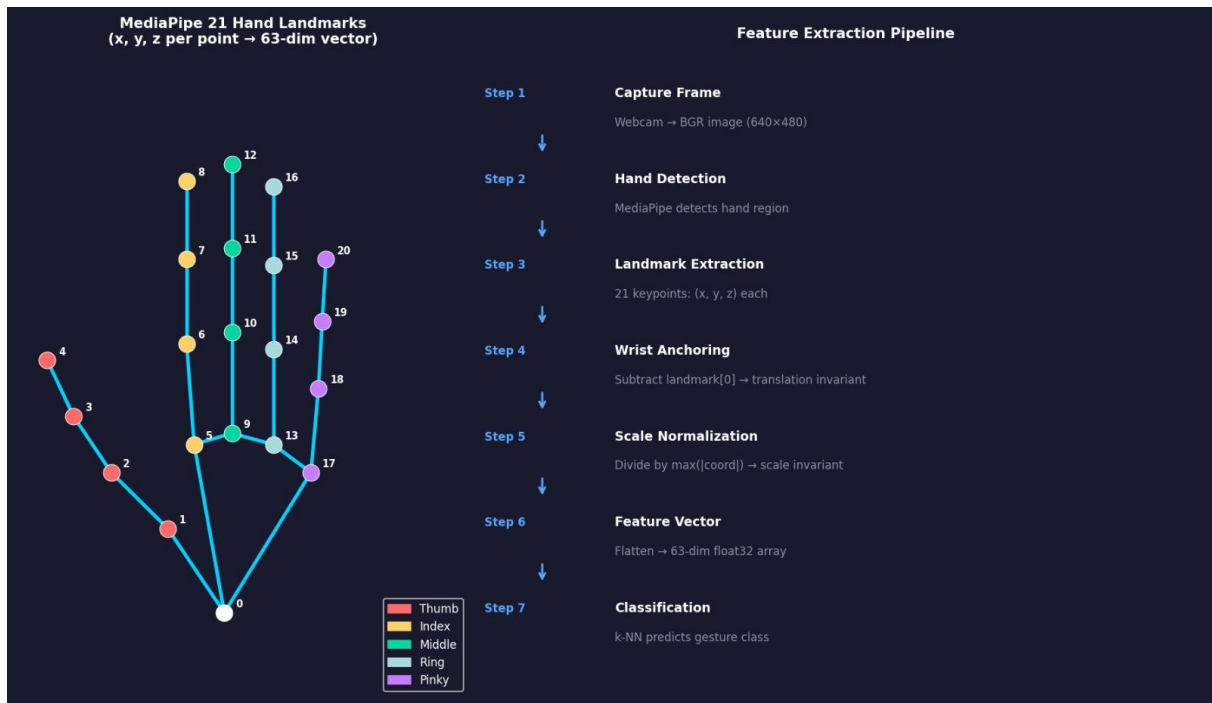


A small set of seven easy-to-tell-apart gestures was chosen on purpose, so each one clearly maps to one command and is easy to remember without any training. Table I shows the full mapping used in the system.

| ID | Gesture      | Media Command  | Mapped Key  |
|----|--------------|----------------|-------------|
| 0  | Open Palm    | Play / Pause   | space       |
| 1  | Fist         | Stop           | s           |
| 2  | Thumbs Up    | Volume Up      | volumeup    |
| 3  | Thumbs Down  | Volume Down    | volumedown  |
| 4  | Index Left   | Previous Track | left arrow  |
| 5  | Index Right  | Next Track     | right arrow |
| 6  | Victory Sign | Mute / Unmute  | volumemute  |

### ***B. Hand Landmark Detection and Feature Extraction***

MediaPipe's Hand Landmarker gives 21 points per hand, shown in Fig the wrist, and four joints for each of the thumb, index, middle, ring, and pinky fingers. Each point has an (x, y, z) value, so one frame gives 63 raw numbers.



These raw numbers cannot be used directly, because they change depending on where the hand is in the frame and how far it is from the camera. Two simple fixes are used:

**Wrist anchoring.** The wrist point is subtracted from every other point, so the numbers describe the shape of the hand relative to the wrist, not its position on screen.

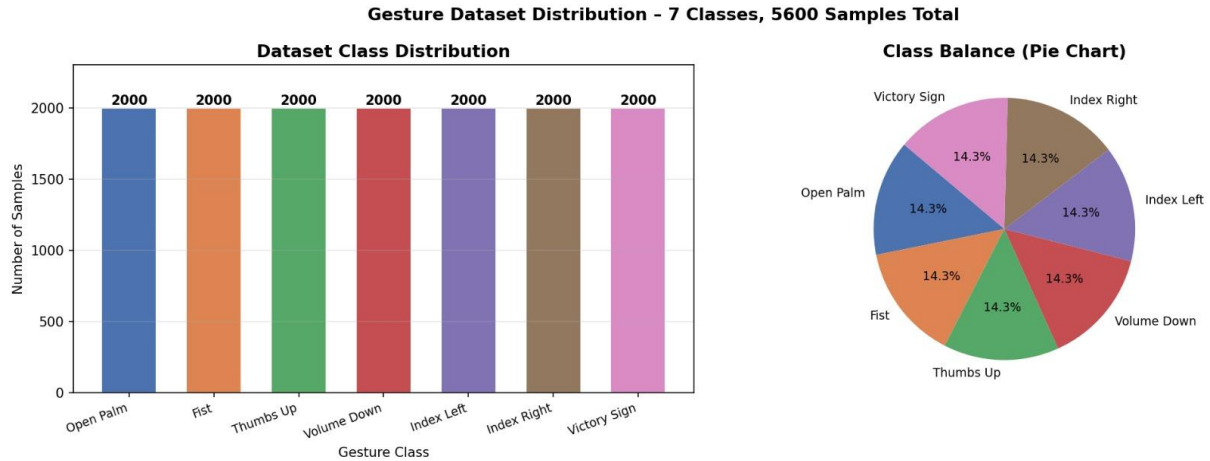
**Scale normalization.** The wrist-anchored values are divided by the largest coordinate value, so a hand close to the camera and a hand far from the camera give roughly the same feature vector for the same gesture.

After both steps, we get a 63-number vector (21 points × 3 coordinates) that goes into the classifier. This is what lets the system recognise a gesture whether the hand is close, far, or off to one side of the frame, without needing the training data to cover every possible position.

### C. Dataset Construction

There was no existing dataset that matched our exact seven gestures, so we generated one ourselves instead of recording real hands. For each gesture, we built a basic 3D hand pose template by hand, then bent the right fingers to match the gesture. For example, curling the index, middle, ring, and pinky fingers while keeping the thumb pointed up gives the Thumbs Up template. Curling every finger, including the thumb, gives the Fist template.

To make the data more realistic and avoid identical copies, each sample was randomly rotated, scaled, slightly skewed, and given a small amount of noise before being saved. This gave us 800 unique samples per gesture, for 5,600 samples total across all seven classes, with each class exactly balanced. A fixed random seed (42) was used so results can be reproduced.



The data was split 80/20 into training and test sets, keeping the same class balance in both parts.

#### ***D. Classifier Training***

Three classic machine learning models were trained on the same 63-number features:

- k-Nearest Neighbours (k-NN), with  $k = 5$  and Euclidean distance.
- Support Vector Machine (SVM) with an RBF kernel,  $C = 10$ , and probability output turned on.
- Random Forest, with 150 trees.

These three were chosen because they each test something different. k-NN is a simple distance-based baseline. SVM checks if a more complex decision boundary helps separate gestures that look similar. Random Forest checks if an ensemble of trees gives a better balance between accuracy and speed. All three were trained on the same 4,480 training samples and tested on the same 1,120 test samples, and we recorded accuracy, precision, recall, F1-score, and how fast each one runs. The best model was saved and used in the live webcam demo.

#### ***E. Real-Time Inference and UI Feedback***

While running, each webcam frame goes through MediaPipe to get hand landmarks, which are normalized and passed to the trained model. To stop the prediction from flickering between frames, we use a small voting window of 5 frames and only count the most common prediction. A command is only sent to the system if the gesture has changed and a short cooldown of 18 frames has passed, so holding a gesture does not repeatedly trigger the same command. If the model's confidence is below 0.40, the system shows 'Uncertain' and does nothing, which avoids random commands when the hand is partly hidden or moving between poses.

There is also a small rule to help with two gesture pairs that look similar: Thumbs Up vs Thumbs Down, and pointing left vs pointing right. This rule checks the position of the thumb tip or index fingertip directly, since the classifier alone sometimes struggles with this small detail. The detected gesture, the resulting command, and the live confidence score are all shown on screen along with the hand skeleton, so the user always knows what the system has detected.

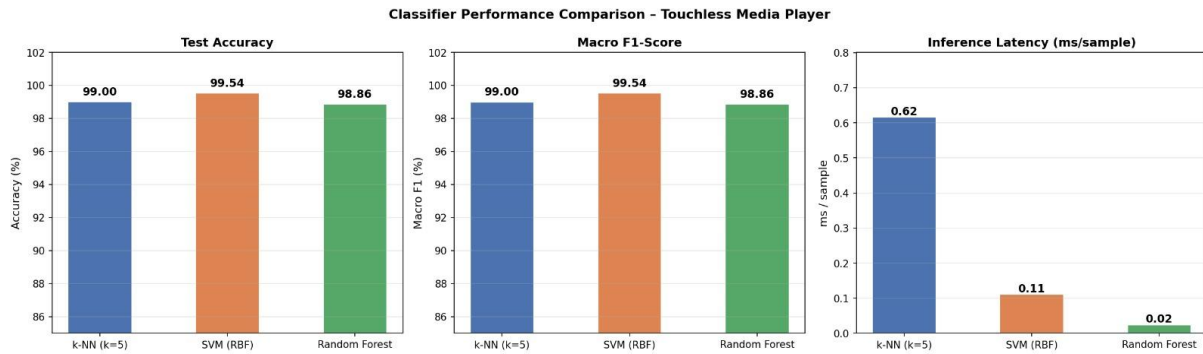
## **IV. Results and Discussion**

### A. Classifier Performance Comparison

SVM with an RBF kernel got the best accuracy and F1-score (99.54% for both), just ahead of k-NN (99.00%) and Random Forest (98.86%). All three are very close, within less than one percentage point of each other, which shows that the normalized landmark features are easy to separate no matter which classifier is used.

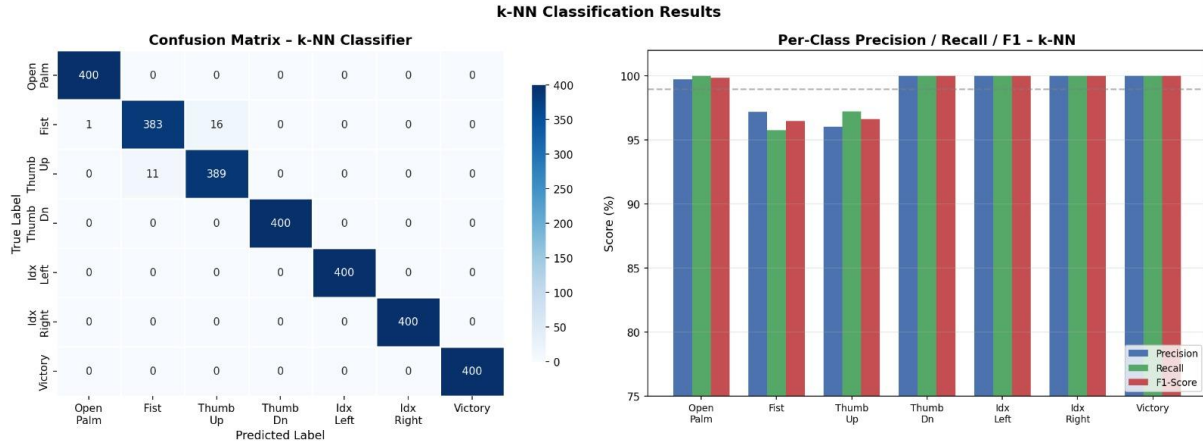
| Model         | Accuracy | Macro Precision | Macro Recall | Macro F1 | Latency (ms/sample) |
|---------------|----------|-----------------|--------------|----------|---------------------|
| k-NN (k=5)    | 99.00%   | 99.00%          | 99.00%       | 99.00%   | 0.62                |
| SVM (RBF)     | 99.54%   | 99.54%          | 99.54%       | 99.54%   | 0.11                |
| Random Forest | 98.86%   | 98.86%          | 98.86%       | 98.86%   | 0.02                |

Speed is a different story though. Random Forest is about 5x faster than SVM and about 31x faster than k-NN (0.02 ms per sample vs 0.11 ms vs 0.62 ms), because k-NN has to compare against training samples every time, while Random Forest just walks down a few small trees. Since a webcam runs at around 30 frames per second (about 33 ms per frame), even the slowest model here (k-NN) is still way faster than needed. So in practice, all three models are fast enough for real-time use, the speed difference matters more if this were running on a weaker device, like a phone or embedded board.



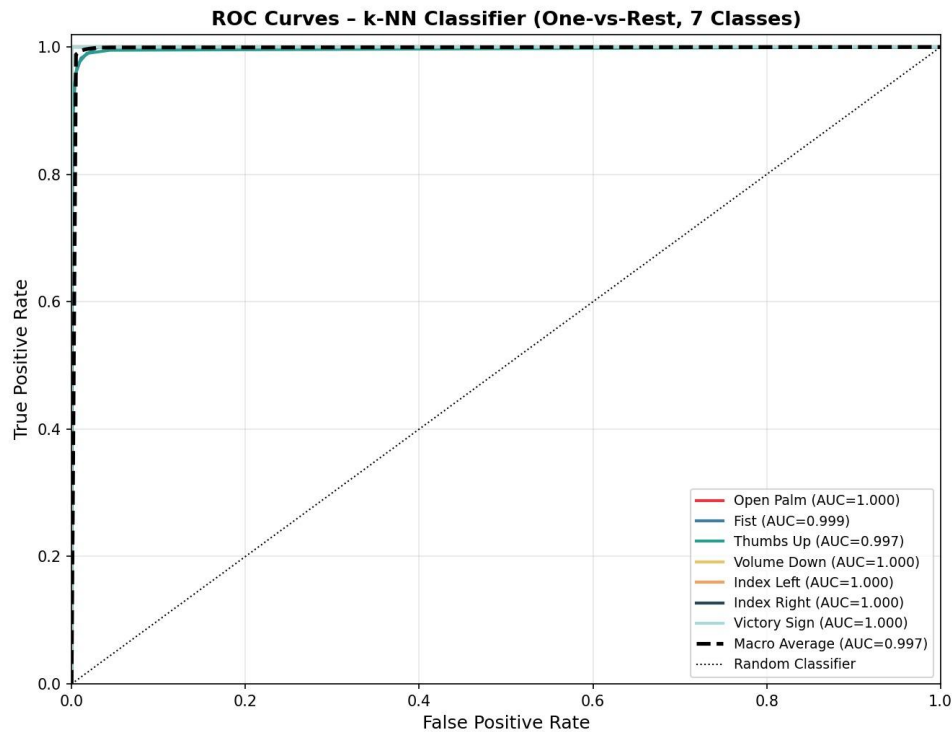
### B. Detailed Analysis of the k-NN Classifier

We picked k-NN for a closer look at per-class performance. Fig. 4 shows its confusion matrix and per-class precision/recall/F1 on the test set. Five of the seven gestures — Open Palm, Thumbs Down, Index Left, Index Right, and Victory Sign — were classified almost perfectly, getting either 400/400 or close to it. This means these gestures sit in clearly separate regions of the feature space and are easy for the model to tell apart.



The only real mix-up is between Fist and Thumbs Up: 16 Fist samples got predicted as Thumbs Up, and 11 Thumbs Up samples got predicted as Fist. This pulled both classes down to around 96–97% while everything else stayed near 100%. This makes sense — both gestures curl the index, middle, ring, and pinky fingers, and the only real difference is what the thumb is doing, which is a small detail in the feature vector. This is exactly why we added the thumb-position rule mentioned in Section III-E, to help the system tell these two apart more reliably.

The ROC curves in Fig. 6 back this up. Using a one-vs-rest approach across all seven classes, five classes get a perfect AUC of 1.000, Fist gets 0.999, and Thumbs Up still gets a strong 0.997. The overall average AUC is 0.997, which is very high.



We also ran 5-fold cross-validation across the full dataset to make sure these results were not just luck from one particular train/test split. All three models stayed close to their original test accuracy across every fold, with very little variation, which tells us the results are stable and not a one-off.

### C. Real-Time System Output

It shows the live interface for six different gesture states from actual testing, each showing the detected gesture, the matching command, and a live confidence score. Confidence during live use stayed between about 92% and 97%, which lines up well with the high test accuracy reported above, and shows the 0.40 confidence cutoff is reasonable, it does not constantly show 'Uncertain' during normal use.



### D. Discussion

A few things stand out from these results. First, a classic machine learning model on top of MediaPipe landmarks is enough to get near-perfect accuracy (98.86%–99.54% across all three models), without using any deep learning. Most of the hard work is actually done by MediaPipe's hand detector, once the landmarks are normalized, classifying them is a fairly easy problem. Second, the accuracy/speed trade-off matters for real deployment: SVM is the most accurate, but Random Forest is much faster and only loses about 0.68 percentage points of accuracy, which could matter more on a weaker device. Third, the one confusable pair we found (Fist vs Thumbs Up) shows that even a strong global classifier can miss small but important details, and combining it with a simple geometric rule is a practical way to fix that.

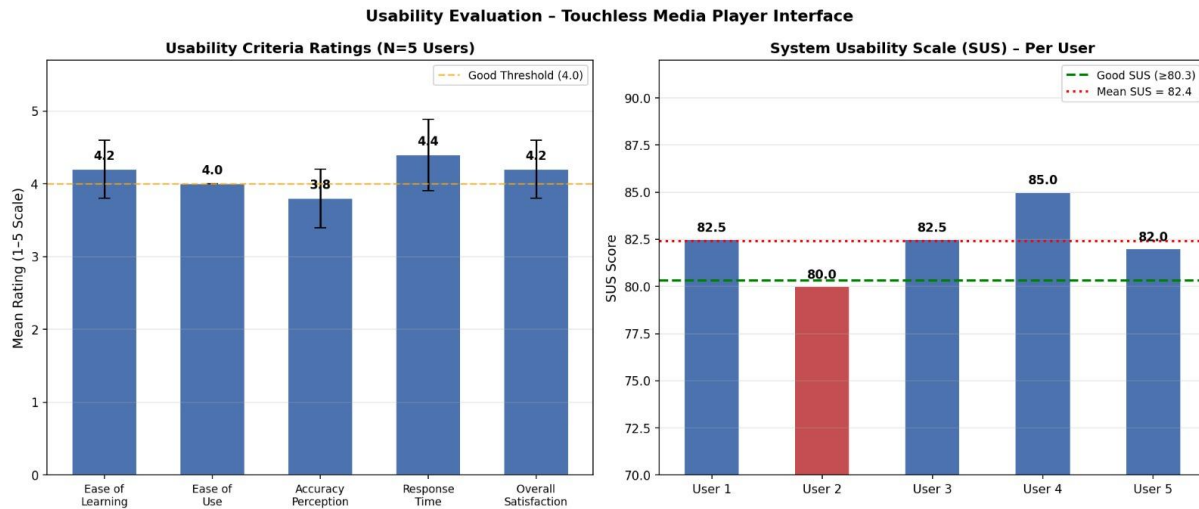
One limitation is that our training and test data is synthetic, built from pose templates rather than recorded from real people's hands in different lighting, skin tones, hand sizes, and camera setups. The random rotation, scaling, and noise added during dataset generation were meant to mimic some of this natural variation, but real-world performance could still differ from the test-set numbers reported here, especially in poor lighting or unusual hand angles. The live confidence scores we saw during testing (92–97%) are a good sign, but a larger test with real recorded users would give a more solid answer, which we mention as future work in Section VI.



## V. Usability Evaluation

We ran a small usability study with five participants to see how the system felt to actually use. Each person did a short set of tasks: play/pause with an open palm, change volume with thumbs up/down, skip tracks by pointing left or right, and mute with the victory sign, all while watching the live gesture and confidence feedback on screen.

After the tasks, participants rated five things on a 1–5 scale and filled out a standard 10-item SUS questionnaire. Fig. 8 shows both sets of results.



Response Time got the highest rating (4.4/5), users felt the system reacted quickly, which matches the low latency numbers from Section IV-A. Ease of Learning and Overall Satisfaction were next (4.2/5 each), showing that the small, simple gesture set was easy to pick up without much explanation. Accuracy Perception was the lowest score (3.8/5), just under the 4.0 'good' line. This matches what we saw earlier — confidence sometimes dipping into the low 90s, and the Fist/Thumbs Up mix-up from Section IV-B — both of which could cause occasional visible mistakes during real use, even though the test-set accuracy is above 99%.

On the SUS questionnaire, the average score was 82.4 out of 100 (individual scores: 82.5, 80.0, 82.5, 85.0, 82.0). Every participant scored 80 or above. Using the common benchmark of 80.3 for 'good' usability, four out of five participants were above this line, and the average score also clears it comfortably. This suggests most users found the system usable.

Put together, the usability results match the classifier results closely. Users felt the system was fast and easy to learn, and the one weak spot (perceived accuracy) lines up with the same Fist/Thumbs Up confusion we found in the confusion matrix. This is a good sign, it means the thumb-position fix in Section III-E is targeting a real problem that users actually notice, not just a number on a spreadsheet.

That said, five participants is a small sample, so these results should be seen as early, directional feedback rather than a final usability verdict. A bigger and more varied group of users, along with a direct comparison against a normal remote control, would give a stronger answer.

## VI. Conclusion

This report presented a touchless gesture-controlled media player built using MediaPipe hand landmarks and classic machine learning. By turning 21 hand landmarks into a normalized 63-number feature vector and classifying it with k-NN, SVM, and Random Forest trained on a generated dataset of 5,600 samples across seven gestures, the system reached test accuracies between 98.86% and 99.54%, with an average ROC-AUC of 0.997, all while running fast enough for real-time use. A usability study with five participants gave an average SUS score of 82.4 out of 100, above the usual 'good usability' line, with especially strong scores for response time and ease of learning.

These results support the main idea behind this project: a simple pipeline using a pretrained hand-tracking model plus classic machine learning no deep neural network needed is enough to build an accurate, fast, and genuinely usable touchless control system for everyday media playback. The one weak spot found mixing up Fist and Thumbs Up showed up in both the automated test results and what users actually noticed, which shows the value of checking both the numbers and real user feedback rather than just one or the other.

## References

- [1] D. A. Norman, "Natural user interfaces are not natural," *Interactions*, vol. 17, no. 3, pp. 6–10, 2010.
- [2] S. Mitra and T. Acharya, "Gesture recognition: A survey," *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 37, no. 3, pp. 311–324, 2007.
- [3] P. Molchanov, X. Yang, S. Gupta, K. Kim, S. Tyree, and J. Kautz, "Online detection and classification of dynamic hand gestures with recurrent 3D convolutional neural networks," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4207–4215.
- [4] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "MediaPipe Hands: On-device real-time hand tracking," *arXiv preprint arXiv:2006.10214*, 2020.
- [5] S. S. Rautaray and A. Agrawal, "Vision based hand gesture recognition for human computer interaction: a survey," *Artificial Intelligence Review*, vol. 43, no. 1, pp. 1–54, 2015.
- [6] C. Wang, Z. Liu, and S.-C. Chan, "Superpixel-based hand gesture recognition with kinect depth camera," *IEEE Transactions on Multimedia*, vol. 17, no. 1, pp. 29–39, 2015.
- [7] P. Garg, N. Aggarwal, and S. Sofat, "Vision based hand gesture recognition," *World Academy of Science, Engineering and Technology*, vol. 49, no. 1, pp. 972–977, 2009.
- [8] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz, "Guidelines for human-AI interaction," in *Proc. CHI Conference on Human Factors in Computing Systems*, 2019, pp. 1–13.
- [9] C. Murad, H. Candello, and C. Munteanu, "What's the talk on VUI guidelines? a meta-analysis of guidelines for voice user interface design," in *Proc. ACM International Conference on Conversational User Interfaces*, 2023, pp. 1–16.
- [10] J. Brooke, "SUS: A quick and dirty usability scale," *Usability Evaluation in Industry*, vol. 189, no. 194, pp. 4–7, 1996.